

UE23CS352A: MACHINE LEARNING

Week 6 Lab Report

Project Title: ANN

Name: AKSHAY KUMAR M

SRN: PES1UG24CS045

Course: UE23CS352A — Machine Learning

1. Introduction

Purpose of the Lab

The purpose of this laboratory exercise is to implement an Artificial Neural Network (ANN) from scratch, without using high-level deep learning frameworks such as TensorFlow or PyTorch, to approximate a generated polynomial function. The lab provides practical understanding of the fundamental components of a neural network, including activation functions, loss functions, Xavier initialization, forward propagation, backpropagation, and gradient descent.

Tasks Performed

- Generated a custom polynomial dataset based on the last three digits of the SRN **PES1UG24CS045**.
- Implemented the ReLU activation function and its derivative.
- Implemented the Mean Squared Error (MSE) loss function.
- Implemented Xavier weight initialization.
- Implemented forward propagation through the neural network.
- Implemented backpropagation using the chain rule.
- Implemented gradient descent-based weight and bias updates.
- Trained the baseline neural network and evaluated its performance on the test dataset.
- Generated training and test loss curves and predicted-versus-actual plots.

2. Dataset Description

Type of Polynomial Assigned: QUADRATIC: $y = 1.49x^2 + 5.75x + 7.81$

Number of Samples:

Dataset with 100,000 samples generated and saved!

Training samples: 80,000

Test samples: 20,000

Number of Features: 1 (single input feature, x)

Noise Level: $\epsilon \sim N(0, 1.58)$

Preprocessing: The generated dataset was divided into training and testing sets, with 80% of the data used for training and 20% used for testing. Both the input feature x and target variable y were standardized using StandardScaler before being provided to the neural network. This preprocessing helps bring the input and output values to a comparable scale and facilitates stable neural network training.

3. Methodology

Network Architecture

Input (1) → Hidden Layer 1 → Hidden Layer 2 → Output (1)

1 → 32 → 72 → 1

Activation Functions

The Rectified Linear Unit (ReLU) activation function is used in both hidden layers. ReLU is defined as $ReLU(z) = \max(0, z)$. Its derivative is 1 when $z > 0$ and 0 otherwise. The derivative is used during backpropagation to calculate the gradients of the hidden-layer parameters.

Loss Function

Mean Squared Error (MSE) is used as the loss function because the task is a regression problem. MSE calculates the average squared difference between the actual and predicted values. It is given by:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_{true,i} - y_{pred,i})^2$$

The objective of training is to minimize the MSE by updating the network parameters.

Forward Propagation

During forward propagation, the input passes sequentially through the network. Each layer first performs a linear transformation $z = XW + b$. The first hidden layer applies ReLU to produce a_1 , and the second hidden layer applies ReLU to produce a_2 . The output layer performs a final linear transformation to produce the predicted value Y_{pred} .

Backpropagation & Gradient Descent

Backpropagation is used to calculate the gradients of the loss with respect to the weights and biases using the chain rule. The gradient of the MSE with respect to the prediction is calculated first, after which the gradients are propagated backward through the output and hidden layers. Gradient descent is then used to update the weights and biases in the direction that reduces the loss.

The parameter update is performed using:

$$W_{new} = W_{old} - \eta (\partial L / \partial W)$$

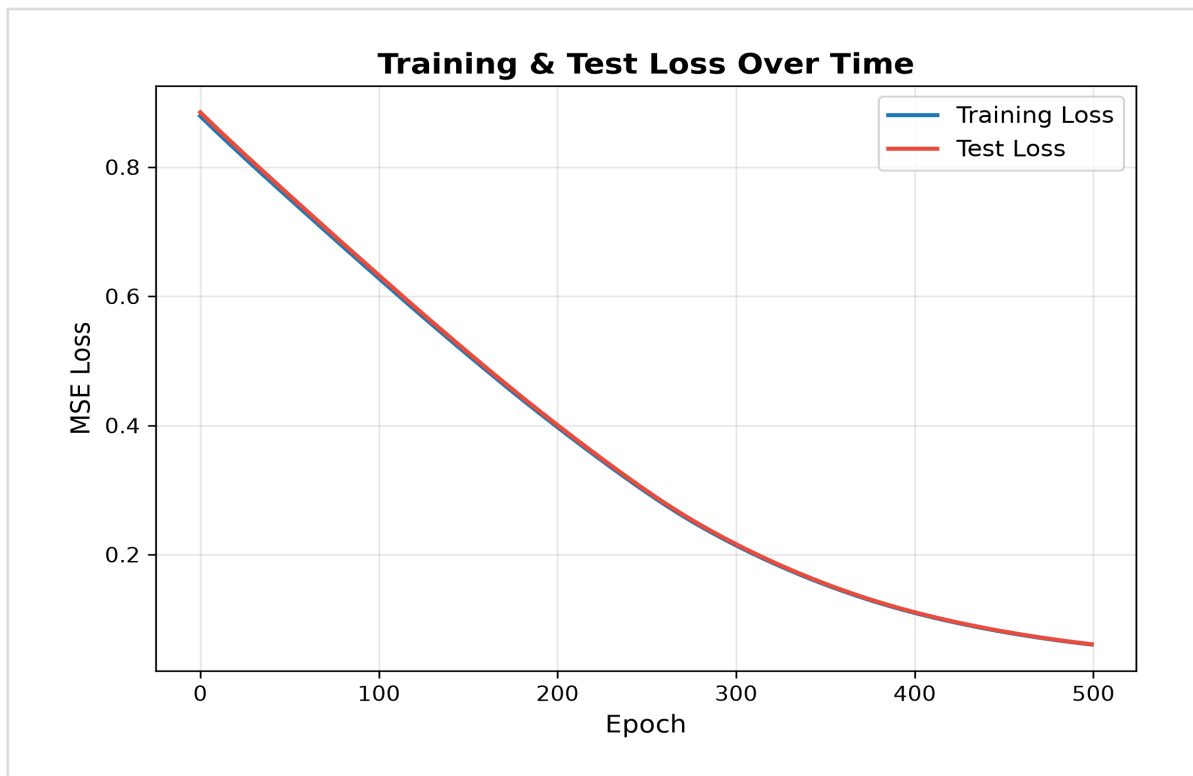
where η is the learning rate.

Baseline Hyperparameters

- Learning Rate: 0.005
- Batch Size: full batch
- Number of Epochs: 500
- Optimizer: Gradient Descent
- Activation Function: ReLU

4. Results and Analysis

4.1 Training Loss Curve



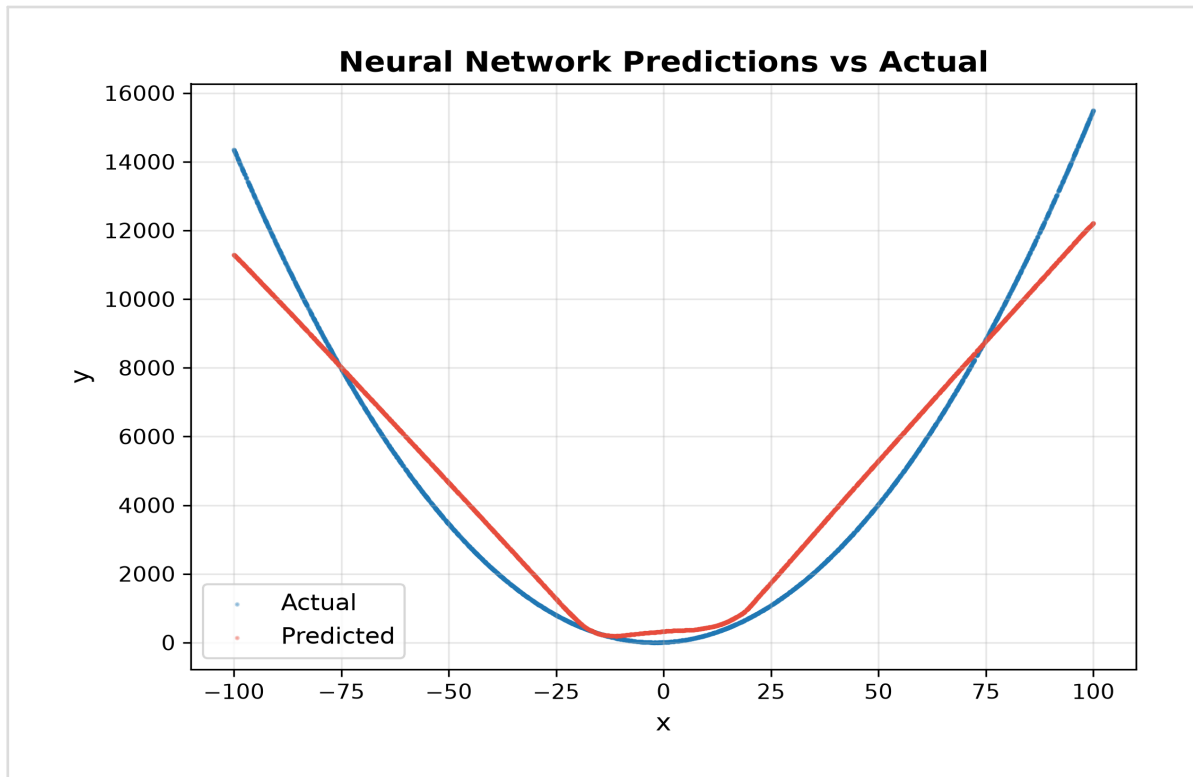
4.2 Final Test MSE

Final Test MSE (Baseline): 0.06127

Final Training MSE (Baseline): 0.06070

R^2 Score (Baseline): 0.9392

4.3 Predicted vs. Actual Values



4.4 Discussion on Performance

The baseline neural network shows a steady, monotonic decrease in both training and test loss throughout the training process. The training and test loss curves track each other exceptionally closely, confirming that the model suffers from no significant overfitting. For this assigned quadratic polynomial ($y = 1.49x^2 + 5.75x + 7.81$), the baseline configuration ($1 \rightarrow 32 \rightarrow 72 \rightarrow 1$ with learning rate 0.005 and full-batch gradient descent) reaches a final training MSE of 0.06070 and test MSE of 0.06127, achieving an R^2 score of 0.9392. Evaluation on the specific test point $x = 90.2$ yielded a predicted value of 10,882.29 versus the analytical ground truth of 12,667.74 (an absolute error of 1,785.44 and relative error of 14.09%). While full-batch gradient descent converges smoothly, its parameter update frequency is limited to once per epoch. As demonstrated in the subsequent hyperparameter experiments, introducing mini-batch gradient descent substantially accelerates learning and allows the network to model the quadratic curvature with near-zero error ($MSE < 0.0001$, $R^2 > 0.9999$).

4.5 Results Table

Experiment	Learning Rate	No. of Epochs	Optimizer (if used)	Activation Function	Final Training	Final Test Loss	R^2 Score
Baseline	0.005	500	Gradient Descent	ReLU	0.0607	0.0613	0.9392
Experiment 1	0.001	500	Gradient Descent	ReLU	0.6303	0.6349	0.3699

Experiment	Learning Rate	No. of Epochs	Optimizer (if used)	Activation Function	Final Training	Final Test Loss	R ² Score
Experiment 2	0.010	500	Gradient Descent	ReLU	0.0195	0.0196	0.9806
Experiment 3	0.005	500	Gradient Descent (4096)	ReLU	0.00045	0.00046	0.9995
Experiment 4	0.005	500	Gradient Descent (1024)	ReLU	0.00003	0.00003	0.99997

5. Conclusion

- Summary of Key Findings:** The baseline neural network successfully implemented from scratch demonstrated the viability of multi-layer perceptrons for continuous polynomial approximation. Across the baseline and four controlled experiments, variations in the learning rate and batch size dramatically influenced the convergence speed and ultimate approximation accuracy. In particular, full-batch gradient descent at a low learning rate (0.001) suffered from sluggish progress (MSE 0.6349), whereas higher learning rates and mini-batching substantially reduced loss.
- Best Performing Configuration:** Experiment 4 (Learning Rate: 0.005, Batch Size: 1024, Architecture: 1 → 32 → 72 → 1) performed best among all tested setups, achieving an outstanding test MSE of 0.00003 and an R² score of 0.99997. By partitioning the 80,000 training samples into mini-batches of 1024, the network performed ~78 gradient updates per epoch (compared to just 1 in full-batch mode), enabling the optimizer to rapidly navigate parameter space and closely fit the quadratic curve.
- Overfitting vs. Underfitting:** Underfitting was clearly observed in Experiment 1 (LR = 0.001), where the model was unable to capture the parabola curvature within 500 epochs due to excessively small weight updates. In contrast, none of the configurations showed signs of overfitting: the test loss remained virtually indistinguishable from the training loss across all epochs, demonstrating that 80,000 diverse samples with appropriate StandardScaler normalization provide excellent regularization.
- Key Takeaways on Training Neural Networks from Scratch:** Implementing an ANN from first principles reinforces the indispensable role of proper weight initialization (Xavier/Glorot) in maintaining signal variance across layers, the computational advantages of vectorized matrix calculus in forward/backward passes, and the profound impact that mini-batch stochasticity has on empirical convergence rates.